Технический анализ ветки `feat/segmentation-dataset`

Репозиторий: DmitriyZhariy/USV-Obstacle-Detection

Для ВКР: «Разработка программного модуля обнаружения и пространственной локализации надводных препятствий для БНСУ»

---

1. Общее описание ветки

Назначение

Ветка `feat/segmentation-dataset` решает задачу **формирования и подготовки обучающего датасета для сегментации** в рамках проекта обнаружения препятствий для беспилотного надводного судна (БНСУ, англ. USV). Согласно README репозитория, цель проекта — реализация исследовательского пайплайна, обрабатывающего данные со стереокамер и датчиков (IMU, GPS) для построения локальной карты препятствий. С добавлением нового коммита от 03.04.2026 ветка расширила свои задачи: помимо статической сегментации отдельных изображений, реализован инженерный фундамент для **видео-паноптической сегментации (VPS)** — задачи, требующей временного контекста и сохранения идентификаторов объектов (Track-ID) между кадрами.

Конкретно в ветке выполняются:

- Извлечение и предобработка кадров из собственных видеозаписей
- Улучшение качества изображений алгоритмическими и нейросетевыми методами
- Маскирование частей корпуса судна, попадающих в кадр
- Конвертация публичного датасета MaSTr1325 в формат YOLO-seg
- Обучение baseline-модели YOLOv8s-seg и её инференс на видео
- Подготовка калибровочного набора для краудсорсинговой разметки
- Генерация плотных последовательных клипов (5 FPS, ±5 сек контекста) для разметки в CVAT командой из 11 аннотаторов
- Автоматизация учёта датасета через синхронизацию с Excel-реестром

Отличие от ветки `main`

Ветка `main` зафиксирована на коммите «Add MIT License» от 01.12.2025 и содержит только скелет проекта: пустые директории и заглушку `main.py`. В `feat/segmentation-`

`dataset` добавлены все 11 скриптов предобработки данных и работы с моделью, а также зафиксированы результаты первого обучения.

Аспект	`main`	`feat/segmentation-dataset`
Скриптов в `src/usv/`	0	11
Директории с кодом	пусто	`data_loader/`, `models/`, `utils/`
Зависимости (runtime)	8 пакетов	11 пакетов
Результаты обучения	нет	`reports/models/mastr_baseline/`
OpenCV	`opencv-python`	`opencv-contrib-python` (DNN SuperRes)
Поддержка Excel	нет	`openpyxl`

Почему для БНСУ важна сегментация (и VPS)

Классическая детекция возвращает bounding box, который не описывает нерегулярные границы навигабельной водной поверхности и препятствий в морской среде. Семантическая сегментация на уровне пикселей позволяет точно выделить область воды как «свободного пространства» и передать эти данные в модуль планирования курса. Видео-паноптическая сегментация добавляет временное измерение: каждому объекту присваивается стабильный Track-ID, что необходимо для оценки скорости и траектории движения встречных судов — ключевое условие для задач предотвращения столкновений и автоматической швартовки.arxiv+3

---

2. Архитектура проекта

Структура директорий (актуальная)

```
USV-Obstacle-Detection/
├── .gitignore
├── .pre-commit-config.yaml
├── .python-version                  # Python 3.11
├── CITATION.cff
├── LICENSE                          # MIT
├── README.md
├── pyproject.toml                   # uv-зависимости (11 runtime-пакетов)
├── setup.py
├── uv.lock
├── main.py                          # заглушка
│
├── configs/
```

```
│   └── config.yaml                  # пустой (Hydra зарезервирован)
│
├── models/
│   └── .gitkeep
│
├── reports/
│   └── models/
│       └── mastr_baseline/
│           ├── args.yaml            # гиперпараметры YOLOv8
│           └── results.csv         # метрики по 50 эпохам
│
└── src/
    └── usv/
        ├── __init__.py
        ├── data_loader/
        │   ├── __init__.py              # NEW: модульные импорты
        │   ├── extract_frames.py        # шаг 1: извлечение разреж. кадров
(обновлён)
        │   ├── enhance_images_alg.py    # шаг 2: CLAHE / гамма-коррекция
        │   ├── upscale_images_ml.py     # шаг 3: super-resolution EDSR/FSRCNN
        │   ├── apply_ignore_mask.py     # шаг 4: маска носа судна
        │   ├── convert_mastr.py         # MaSTr1325 → YOLO-seg
        │   ├── prepare_calibration.py  # подготовка наборов для разметчиков
        │   ├── generate_sequences.py    # NEW: плотные клипы 5 FPS для VPS
        │   ├── split_long_clips.py      # NEW: разбиение длинных клипов на части
        │   └── update_dataset_log.py   # NEW: синхронизация Excel-реестра
        ├── models/
        │   ├── train_baseline.py
        │   └── predict_video.py
        └── utils/
            └── download_sr_models.py
```

Компоненты работы с датасетом

Модуль `data_loader` теперь содержит 9 скриптов, образующих два параллельных пайплайна:

Пайплайн A — разреженные кадры (для статической сегментации, MaSTr1325-style):

- `extract_frames.py` — извлечение кадров с шагом `interval_sec` (обновлён: поддержка `.mkv`, параметр `rotate_center`, детектирование источника по полному пути к файлу)
- `enhance_images_alg.py`, `upscale_images_ml.py`, `apply_ignore_mask.py`, `prepare_calibration.py`, `convert_mastr.py` — без изменений

Пайплайн B — плотные последовательности (для VPS):

- `generate_sequences.py` — основной класс `SequenceDatasetBuilder`, реализующий восстановление метаданных, плотное семплирование и слияние интервалов
- `split_long_clips.py` — класс `SequenceSplitter`, делящий клипы >30 сек на части `_part1`, `_part2` для CVAT
- `update_dataset_log.py` — синхронизация Excel-реестра с файловой системой через `openpyxl` + `pandas`

Поток данных (обновлённый)

Путь A — публичный датасет (MaSTr1325) → статическая сегментация:

```
data/external/MaSTr1325/images/ + masks/
  → convert_mastr.py (YOLO-seg полигоны)
  → data/processed/mastr_yolo/dataset.yaml
  → train_baseline.py
  → reports/models/mastr_baseline/weights/best.pt
```

Путь B1 — собственные данные → разреженная разметка (статическая):

```
data/raw/{left_cam, right_cam, center_phone}/
  → extract_frames.py (5 сек интервал, v5)
  → enhance_images_alg.py → upscale_images_ml.py
  → apply_ignore_mask.py
  → prepare_calibration.py (11 разметчиков)
  → [ручная разметка в CVAT/Roboflow]
```

Путь B2 — собственные данные → плотные последовательности (VPS): *(новый)*

```
data/interim/labeling_v5_mf_squeezed/  ← отобранные «лучшие» кадры
  → generate_sequences.py
      build_index_mapping()  ← восстановление global_idx→(video, frame)
      extract_sequences()    ← клипы 5 FPS, ±5 сек контекст
  → data/interim/sequent_frames_v5/
      ├── videos/  *.mp4
      ├── frames/  clip_name/0001.jpeg ... NNNN.jpeg
      └── metadata/  clip_name.csv
  → split_long_clips.py (разбиение >150 кадров на _partN)
  → update_dataset_log.py
  → dataset_log_updated.xlsx
  → [ручная разметка в CVAT с Track-ID]
  → [обучение VPS-модели]
```

---

3. Используемые зависимости

Полный список зависимостей (актуальный)

Библиотека	Версия (≥)	Роль
`torch`	2.9.1	Бэкенд для обучения YOLOv8
`torchvision`	0.24.1	Трансформации изображений
`ultralytics`	8.3.235	YOLOv8-seg: обучение и инференс
`opencv-contrib-python`	4.11.0.86	Обработка видео/изображений + DNN SuperRes
`openpyxl`	3.1.5	чтение/запись Excel-реестра датасета
`pandas`	2.3.3	Работа с CSV-метаданными клипов и Excel
`numpy`	2.3.5	Матричные операции с масками
`hydra-core`	1.3.2	Управление конфигурацией через YAML
`matplotlib`	3.10.7	Построение графиков метрик
`scipy`	1.16.3	Научные вычисления (для модуля mapping)
`tqdm`	4.67.1	Прогресс-бары

Dev-зависимости: `pre-commit>=4.5.0`, `ruff>=0.14.7`, `pytest>=9.0.1`, `ipykernel>=7.1.0`

Сравнение зависимостей: ветки и коммиты

Пакет	`main`	`feat`	Причина
`ultralytics`	—	≥8.3.235	YOLOv8-seg
`tqdm`	—	≥4.67.1	Прогресс итерации
`opencv-python`	≥4.11.0.86	заменён	DNN SuperRes
`openpyxl`	—	**≥3.1.5**	Excel-реестр

---

4. Конкретные реализации и анализ кода

Формирование датасета: MaSTr1325 (без изменений)

MaSTr1325 — публичный датасет для обнаружения препятствий на береговых USV. В `convert_mastr.py` реализована конвертация попиксельных масок PNG (классы: `0=Obstacle, 1=Water, 2=Sky, 4=Ignore`) в формат YOLO-seg с нормализованными

полигонами. Алгоритм Рамера–Дугласа–Пекера с `epsilon = 0.005 * arcLength` упрощает контуры; порог `contourArea < 50` отфильтровывает шум. Разбиение train/val — 80%/20%.semanticscholar+1

Класс SequenceDatasetBuilder: восстановление метаданных и плотное семплирование

`generate_sequences.py` реализует класс `SequenceDatasetBuilder` с тремя методами, каждый из которых решает отдельную инженерную задачу.

Шаг 1: `build_index_mapping(original_interval_sec=5)`

Проблема: «squeezed»-папка содержит кадры с именами вида `left_00123.jpg`, где `00123` — глобальный индекс из прежнего пайплайна `extract_frames.py`. Чтобы извлечь видеоконтекст вокруг этих кадров, нужно знать их точное положение в исходных видеофайлах. Метод симулирует оригинальное извлечение (тот же шаг, тот же порядок файлов) и строит маппинг:

```
self.mapping[global_idx] = {
    'video_path': vid_file,   # путь к исходному .mp4/.avi
    'frame_idx': i,           # позиция в видео (номер кадра)
    'prefix': prefix,         # 'left' / 'right' / 'center'
    'fps': fps                # реальный FPS видеофайла
}
```

Шаг 2: `merge_intervals(intervals)`

Если несколько «опорных» кадров находятся в одном видеофайле и их контекстные окна (±5 сек = ±150 кадров при 30 FPS) пересекаются, функция слияния объединяет их в единый интервал:

```
def merge_intervals(self, intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for current in intervals[1:]:
        prev = merged[-1]
        if current[0] <= prev[1]:      # пересечение или смежность
            prev[1] = max(prev[1], current[1])
        else:
            merged.append(current)
    return merged
```

Этот алгоритм гарантирует отсутствие дублирования кадров в разных задачах разметки и обеспечивает непрерывность Track-ID для длинных сцен. Временная сложность — O(nlogn) за счёт сортировки.

Шаг 3: `extract_sequences()`

Для каждого слитого интервала извлекаются кадры с шагом `frame_step = fps / target_fps` (например, каждый 6-й кадр при исходных 30 FPS и целевых 5 FPS). Результат сохраняется тройкой:

- `videos/clip_name.mp4` — MP4 для воспроизведения в CVAT
- `frames/clip_name/0001.jpeg ... NNNN.jpeg` — именованная JPEG-последовательность
- `metadata/clip_name.csv` — таблица `(local_seq_idx, original_video, original_frame_idx)`

Именование клипов: `{prefix}_{video_stem}_{seq_id:04d}`, например `right_MOVI0017_0001`.

Разбиение длинных клипов: SequenceSplitter

Класс `SequenceSplitter` в `split_long_clips.py` работает постфактум с уже созданной структурой датасета. Пороговое значение: 30 сек × 5 FPS = 150 кадров:

```
self.max_frames = max_seconds * target_fps  # = 150
```

При разбиении клипа на `N` частей каждой части (`_part1`, `_part2`, ...) присваивается новая нумерация кадров с 1, создаётся новый MP4 и CSV. Исходные файлы удаляются **только после успешного создания всех частей** (`shutil.rmtree` вызывается в конце метода `_process_clip`), что защищает от потери данных при прерывании. Скрипт идемпотентен: клипы, уже содержащие `_part` в имени, пропускаются:

```
if "_part" in csv_path.stem:
    continue
```

Excel-реестр: update_dataset_log.py

`update_dataset_log.py` решает задачу синхронизации файловой системы и ручного реестра аннотаций. Функция `update_dataset_log` реализует следующую логику:

- Из папки `metadata/` читаются все `.csv`-файлы; длительность каждого клипа вычисляется как `len(df) / 5.0` секунд
- Для клипов `_part1` и непарных клипов переносятся существующие метаданные (класс объектов, сложность сцены, комментарии) из старого Excel-файла по имени

базового клипа
- Для `_part2`, `_part3`, ... — создаётся пустая строка (только имя и длительность), так как эти части не имели аналогов в прежнем реестре
- Строгий порядок столбцов зафиксирован в коде (`cols`), что обеспечивает совместимость с ручными таблицами команды

Реестр включает столбцы: `название отрывка`, `брать ли в финальную выборку?`, классы объектов (`Water`, `Sky`, `Land`, `Piers`, `Bridge`, `Vessel`, `Buoy`, `LandingMark`, `BridgeLight`, `Other`), `Сложность`, `Длительность`, `комментарий`.

Обновление extract_frames.py

В новом коммите `extract_frames.py` получил три улучшения:

1. **Поддержка `.mkv`** — добавлен в список расширений (`*.mkv`)
2. **Параметр `rotate_center: bool = True`** — поворот стал опциональным (для сценариев, где телефон уже ориентирован правильно)
3. **Детектирование источника по полному пути** (`str(vid_file).lower()`) вместо имени файла — исправляет ошибку, когда видеофайл лежал в папке `center_phone/`, но имя файла не содержало слова `phone` или `center`

Метрики качества и результаты обучения

Из `reports/models/mastr_baseline/results.csv` задокументированы метрики по 50 эпохам:

Эпоха	mAP50(M)	mAP50-95(M)	Precision(M)	Recall(M)	val/seg_loss
1	0.836	0.601	0.890	0.784	1.516
10	0.773	0.607	0.751	0.775	1.698
20	0.900	0.726	0.933	0.843	1.051
30	0.903	0.751	0.918	0.870	0.951
40	0.927	0.778	0.950	0.877	0.916
50	**0.937**	**0.797**	**0.947**	**0.896**	**0.884**

mAP50(M) = 0.937 при пороге IoU 0.5 — модель корректно сегментирует воду и препятствия в 93.7% случаев на валидации MaSTr1325. Устойчивое снижение val/seg_loss (1.516 → 0.884) свидетельствует об отсутствии переобучения.

---

5. Анализ развития ветки

История коммитов (полная)

Дата	SHA (короткий)	Содержание
27.11.2025	`343a9d69`	Инициализация структуры проекта
01.12.2025	`1f865bdc`	Миграция на uv, реструктуризация `src/`
01.12.2025	`db117268`	MIT License ← точка отхода `main`
09.12.2025	`838fa3f3`	Извлечение кадров с фиксом ориентации
10.12.2025	`c870ec46`	Baseline YOLOv8: конвертер MaSTr + обучение + инференс (+656 строк)
10.12.2025	`7c1885c5`	Пайплайн super-resolution (EDSR/FSRCNN)
10.12.2025	`61313eb6`	Маскирование частей судна bitwise_and
14.12.2025	`77203d3a`	Подготовка кадров для тест-разметки (calibration script)
03.04.2026	`95538df4`	VPS-пайплайн: generate_sequences + split_long_clips + update_dataset_log

Разрыв между коммитами составил **~3,5 месяца** (14.12.2025 → 03.04.2026). Новый коммит стал крупнейшим по числу добавленных файлов (+4 новых скрипта, 480 строк) и ознаменовал качественный переход от статической сегментации к видео-задачам.

Текущий статус ветки (обновлённый)

Завершено:

- Полный пайплайн предобработки собственных видеозаписей (7 скриптов)
- Конвертер MaSTr1325 → YOLO-seg
- Baseline-обучение YOLOv8s-seg на MaSTr1325 (50 эпох, mAP50 = 0.937)
- Скрипт инференса на видео
- Инструментарий для плотного семплирования и подготовки VPS-датасета
- Автоматизация Excel-реестра для команды из 11 аннотаторов

В процессе / не завершено:

- Собственный датасет ещё не размечен — скрипты `prepare_calibration.py` и `generate_sequences.py` лишь готовят данные для передачи в CVAT
- `configs/config.yaml` — по-прежнему пустой файл
- VPS-модель не выбрана и не настроена (только инфраструктура для данных)
- `main.py` — остаётся заглушкой

Проблемы и ограничения

Специфика водной среды:
Водная поверхность остаётся одной из наиболее сложных сред для компьютерного зрения — блики, туман, волнение деформируют текстуру поверхности. Для VPS-задачи добавляется специфическая сложность: Track-ID объектов может прерываться при частичной окклюзии (волна закрывает буй) или при временном выходе объекта за край кадра. Ни один из текущих скриптов не содержит логики обработки прерываний трека.box.vicos

Дисбаланс классов:
В MaSTr1325 водная поверхность занимает подавляющую долю пикселей; явная балансировка классов (`cls_weights`) в `args.yaml` не задана. Для VPS-датасета на речных данных дисбаланс может усугубиться: пролёты под мостами и у причалов создают сцены с преобладанием классов `Bridge` и `Piers` (присутствующих в реестре `update_dataset_log.py`).arxiv

Вычислительная стоимость пайплайна:
`build_index_mapping()` перечитывает метаданные всех видеофайлов (без декодирования), но `extract_sequences()` выполняет реальное декодирование: 480 строк кода добавлены к пайплайну, который на CPU будет работать многие часы. Обучение baseline также велось на CPU (`device='cpu'` , ~815 сек/эпоха).

Смещение домена (domain gap):
Baseline обучен на MaSTr1325, снятом в прибрежных условиях Средиземноморья; целевой датасет — внутренние водоёмы России. Различие в цвете воды, типах берегов и освещении гарантирует деградацию метрик без доменной адаптации или fine-tuning.

Перспективы интеграции

Интеграция в `main` :
После завершения ручной разметки в CVAT и дообучения моделей ветка готова к мёржу. Ключевые артефакты для переноса: все 9 скриптов `src/usv/data_loader/` , обновлённый `pyproject.toml` (11 зависимостей), структура выходных директорий (`videos/` , `frames/` , `metadata/`).

Интеграция с `feat/synchronization` :
Ветка `feat/synchronization` работает над синхронизацией стереокамер, IMU и GPS. После её завершения сегментационные маски из `feat/segmentation-dataset` станут входом для модуля 3D-реконструкции: пиксели класса `obstacle` проецируются в 3D-пространство через откалиброванные стереопары. Трековые идентификаторы из VPS-датасета обеспечат временну́ю согласованность карты препятствий — именно то, что требуется для надёжной автономной навигации.

Следующие шаги по модели:
Для перехода от статической YOLOv8-seg к полноценной VPS-модели необходимо выбрать архитектуру с поддержкой временного контекста (например, Video K-Net, Mask2Former с временными слоями или DVIS). Сформированный пайплайн `generate_sequences.py` уже генерирует данные в нужном формате: `frames/0001.jpeg ... NNNN.jpeg` + `metadata/*.csv` с оригинальными индексами кадров — стандартный вход для большинства VPS-фреймворков.